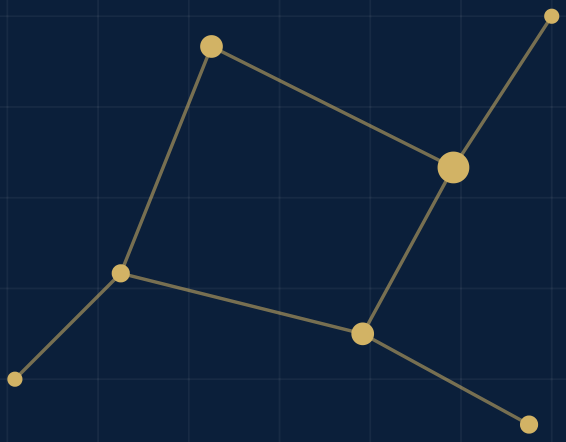
SRM UNIVERSITY · AGENTIC AI & FULL-STACK WORKSHOP



05 — DAY 1 · PROMPTS EXPLAINED

Prompt Guide

Every CampusBite prompt, explained line by line

Why each prompt in the CampusBite handbook is written the way it is: what every line does, what to check in the answer, and how to write your own prompts for the hackathon.

PROMPTS	USE WITH	GOAL
15 explained	Guide 2	Write your own

PREPARED BY

Sathish Kumar K.

AI Educator

Learner reference · Companion to Guide 2

November 2026

Contents

Every CampusBite prompt, explained line by line. Follow the sections in order; each one ends with a checkpoint or checklist.

01	About this guide	3
02	What makes a good prompt?	3
03	The five parts of a build prompt	4
04	P1 · Step 1: Plan with AI	4
05	P2 · Step 2: Read tools	6
06	P3 · Step 3: Write tools	7
07	P4 · Step 4: System prompt	9
08	P6 · Step 5: API routes	11
09	P7 · Step 6: Student screen	12
10	P8 · Step 7: Kitchen screen	13
11	P9 · Step 8: Fix build errors	14
12	P10 to P14 · Stretch features	14
13	P15 · Plan your hackathon problem	16
14	Prompt mistakes to avoid	17
15	Prompt checklist	17

01 About this guide

Guide 2 (the CampusBite Learner Handbook) gives you 15 prompts to copy. This guide explains **why each prompt is written the way it is**: what every line does, what to check in the answer, and what goes wrong if you leave a line out.

Read it alongside Guide 2. When you understand these prompts, you can write your own for the Day 2 hackathon.

Part	What you learn
Sections 1 to 3	What makes a good prompt, the five parts, and how to point the AI at files
Sections 4 to 12	The 9 build prompts, Steps 1 to 8, explained line by line
Section 13	The 5 stretch feature prompts
Section 14	The hackathon planning prompt, as a template for your own problem
Sections 15 and 16	Prompt mistakes to avoid, and a one-page checklist

> **Before you start:** have Guide 2 open. The prompt numbers here (P1 to P15) follow the same order as Guide 2.

02 What makes a good prompt?

An AI coding agent is like a very fast new teammate who has never seen your project. It will do exactly what you ask, quickly. If your request is vague, it will guess, and guesses cause bugs.

A good prompt removes the guessing. Compare:

Weak prompt	Strong prompt
"Add a menu feature"	"In `lib/agent/tools.ts` add `getMenu`: reads `menu_items` where stock > 0, optional `veg_only` and `max_price`, returns id, name, price and stock"
The AI picks the file, the name, the filters and the output	You choose all of them; the AI only writes the code
Different students get different code	Everyone gets almost the same code

> **Key idea:** the more decisions you make in the prompt, the fewer mistakes the AI can make.

03 The five parts of a build prompt

Almost every prompt in Guide 2 has the same five parts. Once you see them, the prompts are easy to read and easy to write.

Part	Question it answers	Example from Step 2
1. Where	Which file or folder?	"In `lib/agent/tools.ts`"
2. What	What should be built or changed?	"replace the 3 task example tools with 4 read tools"
3. How	Which pattern should it follow?	"using `defineTool` like the existing examples"
4. Details	Names, inputs, outputs, tables	"getMenu: reads menu_items where stock > 0..."
5. Rules	What must always or never happen?	"Never take a student id as an argument"

Two more parts appear when needed:

- **Limits:** what not to do yet, for example "Do not write code yet"
- **Done when:** how you know it worked, for example "until npm run build passes"

Pointing at files. Guide 2 uses GitHub Copilot's syntax. In Google Antigravity, Codex and Claude Code, use `@` instead.

You want to	GitHub Copilot	Google Antigravity
Show the whole project	`#codebase`	Name the folders: `@lib @app @seed`
Show one file	`#file:lib/agent/tools.ts`	`@tools.ts` (pick it from the list)
Show a terminal error	`#terminalLastCommand`	Select the error, press `Ctrl+L`

The rest of every prompt stays exactly the same in any tool.

04 P1 · Step 1: Plan with AI

MODE Ask (Copilot) or plan (Antigravity)

PROMPT

```
#codebase Read this Next.js starter and seed/campusbite.sql.
I want to turn this starter into CampusBite, an SRM canteen ordering agent.
Students order food in plain English; the AI agent checks the menu, wallet and pickup slots, then
places the order. A kitchen screen shows live orders.
Give me a short build plan: which files to create or change, which agent tools we need, and which
API routes and pages. Do not write code yet.
```

What it does: asks the AI to read the project and the database file, understand the goal, and give a plan. No code is written.

Line in the prompt	Why it is there
`#codebase Read this Next.js starter`	The AI must see the existing code first, or it will invent a new structure
`and seed/campusbite.sql`	The SQL file holds the real table names, columns and the `place_order` function. Without it, the AI guesses names like `food` or `price_inr`
"I want to turn this starter into CampusBite..."	One line of goal. The AI now knows the end result
"Students order food in plain English; the AI agent checks..."	Describes the user and the flow, so the plan includes the right tools
"which files..., which agent tools..., which API routes and pages"	Tells the AI exactly what the plan must contain. A plan without this is often a long essay
"Do not write code yet"	Keeps this step for thinking only. Code written before a plan is agreed is often thrown away

Check the answer: the plan names six tools, a new file `lib/campusbite.ts`, the three API routes, the kitchen page, and says orders should go through the database function, not direct updates.

If you skip this step: the AI starts coding at once and makes its own design choices. You then spend time undoing them.

Technique: plan first. Ask for the plan in a fixed shape (files, tools, routes, pages).

05 P2 · Step 2: Read tools

MODE Agent (Copilot) or default (Antigravity)

PROMPT

```
In #file:lib/agent/tools.ts replace the 3 task example tools with 4 read tools for CampusBite, using defineTool like the existing examples.
- Create lib/campusbite.ts with: export const CURRENT_STUDENT_ID = "S001"; and a helper formatSlot("13:15:00") → "1:15 pm".
- getMenu: reads menu_items where stock > 0. Optional args: category (meal, snack, drink, sweet), veg_only (boolean), max_price (integer). Order by category, then price. Return id, name, category, price, is_veg, stock, description.
- getWalletBalance: no args. Returns name and balance for CURRENT_STUDENT_ID from wallets.
- getPickupSlots: no args. Reads pickup_slots, returns slot_id, time (formatted) and free (capacity - booked), only slots with free > 0.
- getMyOrders: no args. Returns the student's last 10 orders with id, status, total, pickup time and item names (join order_items and menu_items).
Descriptions must start with "Use this when...". Never take a student id as an argument.
```

What it does: builds the agent's "eyes": four tools that only read data.

Line in the prompt	Why it is there
`In #file:lib/agent/tools.ts replace the 3 task example tools`	Says where, and that the old example tools must go. Otherwise the AI may keep them and the agent gets confused
"using defineTool like the existing examples"	Points at a pattern that already works. The AI copies the style instead of inventing one
`CURRENT_STUDENT_ID = "S001"`	One place that decides who the student is. Every tool uses it
`formatSlot("13:15:00") → "1:15 pm"`	An input and output example. Examples are the clearest way to describe a small function
"where stock > 0"	The agent must never offer sold-out food. This rule lives in the query, not in the AI's memory
"Optional args: category (meal, snack, drink, sweet)..."	Listing the allowed values stops the AI from inventing categories like "lunch"
"Order by category, then price"	Gives a stable order, so the agent can pick the cheapest item easily
"Return id, name, category..."	Returns only what the agent needs. Smaller results use fewer tokens and confuse the model less
"no args" for wallet, slots and orders	These tools always work for the logged-in student, so they need nothing from Gemini

Line in the prompt	Why it is there
"free (capacity - booked), only slots with free > 0"	Full slots are hidden, so the agent cannot pick a slot that has no space
"join order_items and menu_items"	Tells the AI how the tables connect, so item names come back instead of only ids
"Descriptions must start with 'Use this when...'"	Gemini reads the description to decide when to call a tool. This wording makes the choice clear
"Never take a student id as an argument"	Security. If Gemini could pass a student id, a user could say "order as S002" and spend someone else's money

Check before you accept: every tool uses `CURRENT_STUDENT_ID`, and `getMenu` has `.gt("stock", 0)` or similar.

> **Test in the app:** "What's veg under Rs 80 today?" The agent should call `getMenu` with `veg_only: true` and `max_price: 80`, and Paneer Butter Masala (sold out) should not appear.

Technique: give the exact names, arguments and return fields. Add one example for anything that formats data.

06 P3 · Step 3: Write tools

MODE **Agent or default**

PROMPT

```
In #file:lib/agent/tools.ts add 2 write tools (writes: true):
- placeOrder: args items (array of { menu_item_id, quantity 1-5 }, at least 1) and slot_id. Call
getSupabase().rpc("place_order", { p_student_id: CURRENT_STUDENT_ID, p_slot_id: slot_id, p_items:
items }). Throw new Error(error.message) on error, else return data.
- cancelOrder: arg order_id. Call rpc("cancel_order", { p_student_id: CURRENT_STUDENT_ID,
p_order_id: order_id }) the same way.
Do not update wallets or stock directly in TypeScript; the database functions do all checks.
The tools array must be exactly: getMenu, getWalletBalance, getPickupSlots, getMyOrders,
placeOrder, cancelOrder.
```

What it does: builds the agent's "hands": two tools that change money, stock and orders.

Line in the prompt	Why it is there
"(writes: true)"	Marks these tools as risky. The starter shows write tools differently in the chat
"quantity 1-5, at least 1"	These limits become a Zod schema. Gemini cannot order 0 or 50 items

Line in the prompt	Why it is there
`rpc("place_order", {...})`	Calls the database function from the SQL file. The exact parameter names (`p_student_id` ...) must match the SQL, so the prompt spells them out
"Throw new Error(error.message) on error"	The database's message ("Not enough balance") goes back to Gemini, which can explain it to the student
"Do not update wallets or stock directly in TypeScript"	The most important line. Without it, the AI often writes 3 separate updates, and a failure halfway leaves the wallet charged with no order
"The tools array must be exactly: ..."	A clear finish line. The AI cannot forget a tool or leave an old one in

Check before you accept: both tools call `rpc(...)`. There is no `.update(` on `wallets` or `menu_items` anywhere in the file.

> **Key idea:** the money and stock rules live in the database, so the AI cannot break them. Your prompt protects that design by saying "do not update directly".

Technique: say what NOT to do when there is an easy wrong way. Name exact function and parameter names when they must match something else.

07 P4 • Step 4: System prompt

MODE Agent or default

This prompt is different: it is a prompt that writes **another prompt**. The text you send becomes the instructions Gemini follows every time a student chats.

PROMPT

```
Rewrite buildSystemPrompt in #file:lib/agent/systemPrompt.ts for CampusBite:
- You are CampusBite, the friendly SRM canteen ordering assistant. The student is
CURRENT_STUDENT_ID.
- Keep the line with today's date. Pickup is today only, 12:00 pm to 2:30 pm.
- Always use tools for menu, prices, stock, wallet and slots. Never invent them.
- Never guess an item id or slot id. Before calling placeOrder, call getMenu and getPickupSlots
in the same turn to get the real ids, even if you listed them earlier.
- Before ordering, check menu, wallet and free slots. If the student gives a budget, fit the
total to it.
- If the student gives a clear request with a budget and a pickup time, choose the best
combination and place the order directly. Only ask when the request is unclear.
- If no pickup time is given, use the earliest free slot. If a time is full, offer the nearest
free time.
- If an item is sold out or the wallet is too low, explain simply and suggest a cheaper or
similar item.
- If the student picks an option you offered (for example "option 1"), place that order directly.
- Only call cancelOrder when the student clearly asks to cancel. Never cancel an order to fix
your own mistake; explain the problem instead.
- If placeOrder fails, read the error, fix the ids and try once more, then explain to the
student.
- After ordering, reply with items, total, pickup time and wallet balance left. Prices like "Rs
70".
- If the kitchen has started an order, explain it cannot be cancelled.
- Keep replies short. Only help with the canteen.
```

What it does: gives the agent its job description: who it is, what it may do, and how it should behave.

Rule	Why it is there
"You are CampusBite, the friendly... assistant"	The role sets the tone and the topic
"Keep the line with today's date"	The starter adds today's date. Without it, Gemini does not know what "today" means
"Pickup is today only, 12:00 pm to 2:30 pm"	Stops orders for tomorrow or 6 pm
"Always use tools... Never invent them"	Stops the agent from making up prices or items from its general knowledge
"Never guess an item id or slot id..."	Fixes a real bug from rehearsal (see below)

Rule	Why it is there
"If the student gives a budget, fit the total to it"	"I have Rs 150" becomes a hard limit, not a suggestion
"choose the best combination and place the order directly. Only ask when unclear"	Without this, the agent asks too many questions and feels slow
"use the earliest free slot... offer the nearest free time"	Gives a clear default, so the agent does not stop to ask
"explain simply and suggest a cheaper or similar item"	A helpful "no" instead of a dead end
"If the student picks an option you offered..."	Fixes "option 1" replies, which confused the agent in rehearsal
"Never cancel an order to fix your own mistake"	Fixes the worst rehearsal bug
"try once more, then explain"	Limits retries, so the agent does not loop forever
"reply with items, total, pickup time and wallet balance left"	The student always sees what happened. Judges look for this too
"Keep replies short. Only help with the canteen"	Stays on topic. "Write me a poem" gets a polite no

> **Rehearsal finding:** without the "never guess" and "never cancel" rules, the agent placed an order with a guessed item id, saw the wrong item, cancelled its own order to try again, and ran out of steps. Every rule in this list exists because something went wrong without it.

Then run P5:

PROMPT

```
In #file:lib/agent/runAgent.ts change MAX_ROUNDS from 5 to 8.
```

Why: one order can need four or five tool calls (wallet, menu, slots, place order, and a retry). Five rounds was too few. Eight gives room, but still stops a runaway loop.

> **Test in the app:** refresh the page, then type "I have Rs 150. Get me lunch with something sweet, pick up at 1:15." Then check the `orders` and `wallets` tables in Supabase.

Technique: write rules as short lines that start with "Always", "Never", "If... then...". Add a rule every time you see the agent make the same mistake twice.

08 P6 · Step 5: API routes

MODE Agent, default or accept-edits

PROMPT

```
Create 3 API routes, following the pattern of #file:app/api/tasks/route.ts:
1. app/api/menu/route.ts GET: all menu_items (including sold out), ordered by category and price.
2. app/api/me/route.ts GET: wallet of CURRENT_STUDENT_ID and their last 5 orders with pickup time
and item names.
3. app/api/kitchen/route.ts GET: orders with status placed, preparing or ready, with student
name, pickup time and items, oldest first.
   POST { order_id, status }: validate with Zod. Only allow the next step: placed → preparing →
ready → collected. Return 409 for any other change.
Add export const dynamic = "force-dynamic" to each. Log errors with console.error.
Also update app/api/health/route.ts to check the menu_items table instead of tasks.
```

What it does: creates the APIs the two screens will use.

Line in the prompt	Why it is there
"following the pattern of app/api/tasks/route.ts"	The tasks route already has the passcode check and error handling. Copying it keeps the new routes safe
"(including sold out)"	The menu screen shows sold-out items with a red badge, unlike `getMenu` for the agent. The prompt makes the difference clear
"last 5 orders with pickup time and item names"	Exactly what the wallet card shows. No more, no less
"oldest first"	The kitchen cooks the oldest order first
"validate with Zod"	Never trust data sent from a screen
"Only allow the next step: placed → preparing → ready → collected"	A business rule: an order cannot jump from "placed" to "collected", or go backwards
"Return 409 for any other change"	409 means "conflict". A clear status code makes errors easy to spot
`export const dynamic = "force-dynamic"`	Stops Next.js from caching the result. Without it, the kitchen screen might show old orders
"Log errors with console.error"	When something fails, you can see why in the terminal or Vercel logs
"update app/api/health/route.ts..."	The health check should test the new tables, because the tasks table will be deleted next

Test in the browser: open `localhost:3000/api/menu` and `localhost:3000/api/kitchen`. You should see JSON.

Technique: number the items when one prompt creates several files. Mention small technical lines (`force-dynamic`) that you know the AI often forgets.

09 P7 · Step 6: Student screen

MODE Agent, default or accept-edits

PROMPT

```
Build the CampusBite student screen:
1. components/WalletOrders.tsx: a dark card showing the wallet balance (Rs) and student name, then "My orders today" with order number, pickup time, items, total and a coloured status badge. Loads /api/me.
2. components/MenuPanel.tsx: today's menu grouped by meals, snacks, sweets, drinks. Each row: green or red veg dot, name, stock ("Sold out" red badge at 0, "Only N left" amber badge at 3 or less), price. Loads /api/menu.
3. Both reload when refreshKey changes AND every 5 seconds, so kitchen updates appear.
4. In app/page.tsx replace TaskList with WalletOrders above MenuPanel on the right. Title "CampusBite · SRM canteen ordering agent", and an "Open kitchen screen" link to /kitchen.
5. In components/AgentChat.tsx set SAMPLE_PROMPTS to: "What's veg under Rs 80 today?", "I have Rs 150. Get me lunch with something sweet, pick up at 1:15.", "Add a chicken biryani too.", "Cancel my latest order."
6. Supabase may return a related row as an object or a one-item array; handle both so item names and pickup times always show.
Use Tailwind. Delete components/TaskList.tsx and app/api/tasks.
```

What it does: builds the screen students see: wallet, orders and the live menu.

Line in the prompt	Why it is there
"a dark card showing the wallet balance..."	Describes the look in words a designer would use: card, badge, colour
"'Sold out' red badge at 0, 'Only N left' amber badge at 3 or less"	Exact rules with numbers. "Show low stock" alone would give a different threshold each time
"Loads /api/me" and "Loads /api/menu"	Connects each component to the API from Step 5
"reload when refreshKey changes AND every 5 seconds"	Two triggers: right after the agent places an order, and when the kitchen changes a status
"replace TaskList with WalletOrders above MenuPanel on the right"	Says where things go on the page
"set SAMPLE_PROMPTS to: ..."	The sample buttons become the test cases for the demo
"Supabase may return a related row as an object or a one-item array"	A real problem seen in rehearsal: item names showed as blank. Naming the problem fixes it in one go

Line in the prompt	Why it is there
"Delete components/TaskList.tsx and app/api/tasks"	Cleans up the old example, so nothing is left half-used

> **Check:** the page shows the wallet (Rs 150 at the start), and Chicken Biryani shows "Only 2 left".

Technique: describe UI with exact rules and numbers. Name known problems in the prompt instead of fixing them later.

10 P8 · Step 7: Kitchen screen

MODE **Agent, default or accept-edits**

PROMPT

```
Create app/kitchen/page.tsx: a dark full-screen kitchen board with 3 columns: New orders,
Preparing, Ready for pickup.
Each order card shows order number, pickup time, student name, items and total, with one button:
"Start preparing" (placed), "Mark ready" (preparing), "Collected" (ready). The button calls POST
/api/kitchen.
Reload /api/kitchen every 3 seconds. Show a green "Live" badge in the header. Handle related rows
that may be an object or an array. Use Tailwind.
```

What it does: builds the screen the kitchen staff use.

Line in the prompt	Why it is there
`Create app/kitchen/page.tsx`	In Next.js, this file path becomes the `/kitchen` page
"3 columns: New orders, Preparing, Ready for pickup"	The layout, in the order the work flows
"with one button: 'Start preparing' (placed)..."	One button per status, with the exact label. Staff never have to choose
"The button calls POST /api/kitchen"	Uses the route from Step 5, which already checks the next-step rule
"Reload every 3 seconds"	Faster than the student screen, because the kitchen needs new orders quickly
"Handle related rows that may be an object or an array"	The same fix as Step 6, repeated so it is not forgotten

Test: open `/kitchen` next to the student screen. Place an order in the chat and watch it appear.

Technique: describe each state and its action as a list. Repeat important fixes in every prompt where they apply.

11 P9 · Step 8: Fix build errors

MODE Ask (Copilot) or default (Antigravity)

After you run `npm run build` and see errors:

PROMPT

```
#terminalLastCommand Explain these errors in one line each and tell me exactly what to change in which file.
```

In Antigravity, select the error text in the terminal, press `Ctrl+L`, then send the same sentence. Or ask the agent to do the whole job: *"Run npm run build and fix all errors until it passes. Explain each fix in one line."*

Part	Why it is there
`#terminalLastCommand`	Gives the AI the real error text. Never describe an error in your own words when you can paste it
"Explain these errors in one line each"	You learn what went wrong, instead of only getting a fix
"tell me exactly what to change in which file"	A clear, small answer you can check
"until it passes" (Antigravity version)	A finish line. The agent keeps going until the build works

Technique: always give the exact error. Ask for an explanation, not only a fix.

12 P10 to P14 · Stretch features

These prompts add new features after the main build. Each one teaches one more idea.

P10: Reorder last

PROMPT

```
In #file:lib/agent/tools.ts add a tool reorderLast (writes: true, no args). It reads the student's latest order that is not cancelled (with its items), picks the earliest free pickup slot, and calls rpc("place_order") with the same items. Description: "Use this when the student asks to order the same as last time." Add it to the tools array.
```

Why it works: it reuses `place_order`, so all the money and stock rules still apply. "that is not cancelled" avoids reordering a cancelled order. The description is given word for word, so Gemini knows exactly when to use it.

Try: "Same as last time, please."

P11: Daily spending limit

PROMPT

```
In #file:lib/agent/tools.ts, before placeOrder calls rpc("place_order"), work out today's total for CURRENT_STUDENT_ID (sum of orders created today that are not cancelled) and the new order total from menu_items prices. If the sum would go above 200, throw new Error("Daily limit of Rs 200 reached. You have Rs X left to spend today."). Add the rule to the system prompt too.
```

Why it works: the rule is checked in code, not only in the prompt, so the agent cannot skip it. The exact error message is given, so the student sees a clear, helpful reply. "Add the rule to the system prompt too" lets the agent warn the student before it even tries.

> **Key idea:** put the rule in code first, then tell the agent about it. A rule only in the system prompt can be ignored by the AI.

P12: Mark sold out

PROMPT

```
Add a "Menu stock" panel at the bottom of app/kitchen/page.tsx that lists menu items with their stock and a "Mark sold out" button. Create app/api/kitchen/stock/route.ts with a POST { menu_item_id } that sets stock to 0 (validate with Zod). The student menu and the agent must stop offering that item.
```

Why it works: one prompt covers the screen, the API and the result you expect. The last sentence is the test: if the agent still offers the item, the feature is not done.

P13: Veg-only preference

PROMPT

```
I want a saved veg-only preference per student.
1. Give me SQL to add a column veg_only boolean not null default false to the wallets table (I will run it in Supabase).
2. Add a tool setVegOnly (writes: true, arg veg_only boolean) that updates it for CURRENT_STUDENT_ID.
3. Make getMenu return only veg items when the student's veg_only is true.
4. Add the preference to the system prompt.
```

Why it works: the first line states the goal. The numbered steps cover all four layers: database, tool, existing tool and system prompt. "I will run it in Supabase" tells the AI not to try to change the database itself.

Try: "I'm vegetarian from now on." Then "What's for lunch?"

P14: Phone screen check

PROMPT

```
Review #file:app/page.tsx and the WalletOrders and MenuPanel components for a 360px wide phone screen. Fix anything that overflows or is hard to tap. Do the same for app/kitchen/page.tsx.
```

Why it works: "360px wide" is an exact size, the width of a common phone. "overflows or is hard to tap" names the two problems to look for. Judges open your Vercel link on a phone, so this matters.

Technique for all stretch prompts: say which existing parts to reuse, give exact messages and numbers, and end with how you will know it works.

13 P15 · Plan your hackathon problem

This is the most important prompt for Day 2. It is the same idea as P1, written as a template.

PROMPT

```
#codebase Read this starter and seed/<your seed file>.sql.
Our problem: <business user> has this problem: <problem in one line>.
Our agent must: <what it should do, in 2 lines>.
Give a short build plan: the read tools and write tools we need (name, arguments, which table),
the business rules to check in code, the API routes and the pages. Do not write code yet.
```

How to fill it in, with PS7 as an example:

Blank	Example (PS7 RouteWise)
`<your seed file>`	`ps7_delivery_dispatch.sql`
`<business user>`	A dispatcher at a city delivery hub
`<problem in one line>`	Assigns deliveries by hand; riders get overloaded and failed deliveries are forgotten
`<what it should do>`	Find pending deliveries by zone, assign them to riders on shift with free capacity, and reschedule failed ones

What to do with the plan:

1. Check every table and column name against your seed file
2. Make sure at least one tool writes to Supabase (the agent rule)
3. Show the plan to your mentor at the first checkpoint
4. Copy the business rules into your `AGENTS.md` file
5. Then write one build prompt per tool, using the five parts from Section 3

Key idea: your Day 2 prompts should look like P2 and P3: exact file, exact names, exact rules. Use them as models.

14 Prompt mistakes to avoid

Mistake	What happens	Do this instead
No file named	The AI edits the wrong file or creates a new one	Start with "In @file..."
No table or column names	The AI guesses names that do not exist	Point at the seed file, or list the columns
Too many features in one prompt	Half-finished code across many files	One feature per prompt
No rules	Security gaps, such as a student id taken from the user	Add "Never..." lines for anything risky
Describing an error in your own words	The AI fixes the wrong thing	Paste or quote the exact error
Accepting without reading	Bugs you cannot explain to the judges	Check the 2 or 3 key points before you accept
Rules only in the system prompt	The AI can ignore them	Put money, stock and limit rules in code or the database
Starting again after a small mistake	Wasted quota and time	Ask for a small fix: "In @file, change X to Y"

15 Prompt checklist

Before you send a build prompt, check:

- [ ] **Where:** I named the file or folder
- [ ] **What:** I said what to build or change in one line
- [ ] **How:** I pointed at an existing pattern to follow
- [ ] **Details:** I gave the names, arguments, return fields and tables
- [ ] **Rules:** I wrote the "always" and "never" lines for anything risky
- [ ] **Limits:** I said what not to do (if needed)
- [ ] **Done when:** I know how I will test it

After the AI answers, check:

- [ ] It changed only the files I asked about
- [ ] The names match my prompt and the database
- [ ] No keys or passwords are in the code
- [ ] I can explain every change in one sentence
- [ ] I tested it in the app before the next prompt

> **Remember:** a good prompt works in any AI coding tool. A tool cannot fix a bad prompt.